International Institute of Information Technology, Bangalore

Software Production Engineering Mini Project Report

Scientific Calculator

Submitted by

Vinay V Bhandare

(MT2025130)

Under the guidance of Prof. B. Thangaraju

Table of Contents

Abstract.....3

Devops.....4

Tools and Technologies Used..... 5

Source Code Management.....6

Jenkins.....7-11

Docker.....12-13

Ansible.....14-15

Source Code.....16-17

Adding Test Cases.....18

Webhooks……19-20

Email Notification through Jenkins..... 21-22

Abstract:

The **Scientific Calculator project** is a software application developed to perform various mathematical and scientific operations such as factorial, square root, logarithmic calculations, and exponentiation. The project integrates **DevOps practices** to enhance the efficiency of application development, testing, and deployment processes. Tools such as **GitHub, Docker, Jenkins, and Ansible** are utilized to automate several stages of the software development lifecycle.

Automation plays an important role in this project by enabling automatic testing and deployment of the application without requiring manual intervention. **Docker** is used to containerize the application, ensuring that it runs consistently across different environments and systems. Additionally, **GitHub and Jenkins** are used to implement **Continuous Integration and Continuous Deployment (CI/CD) pipelines**, which automatically build, test, and deploy the application whenever updates are made to the source code.

By adopting these **DevOps tools and modern development methodologies**, the project becomes more reliable, maintainable, and scalable. It also promotes better collaboration among team members and accelerates the overall development and deployment process.

DevOps:

DevOps is a modern approach to software development that combines the efforts of development and operations teams to work together throughout the entire lifecycle of an application. The primary goal of DevOps is to enable faster and more efficient delivery of applications and services while maintaining reliability and stability.

The main idea behind DevOps is to improve the speed and quality of software delivery by introducing automation, continuous integration, and continuous deployment practices. By streamlining workflows and reducing manual processes, organizations can develop, test, and release software more effectively. DevOps also promotes a culture of collaboration, where teams share responsibilities and work together to solve problems.

Some key benefits of DevOps include:

- **Rapid software delivery:** DevOps allows teams to release updates and new features more frequently and with less delay.
- **Higher software reliability:** Continuous testing and monitoring help detect issues early, improving the overall quality of the software.
- **Stronger team collaboration:** Developers and operations engineers work together closely, reducing communication gaps and improving productivity.
- **Enhanced security practices:** Security measures are integrated into the development pipeline, ensuring safer applications.
- **Efficient scalability:** Automated systems and streamlined processes make it easier to scale applications when demand increases.
- **Better customer satisfaction:** Faster updates and reliable software lead to a better experience for users.
- **Reduced operational costs:** Automation and efficient workflows help minimize the costs associated with slow or error-prone software development processes.

Tools and Technologies Used:

Programming Language: Java 21

Build Tool: Maven 3.x

Testing: Junit 5

CI/CD Pipeline: Jenkins

SCM: GitHub + Webhook(ngrok) + GitSCM Polling

Containerization: Docker

Deployment: Ansible

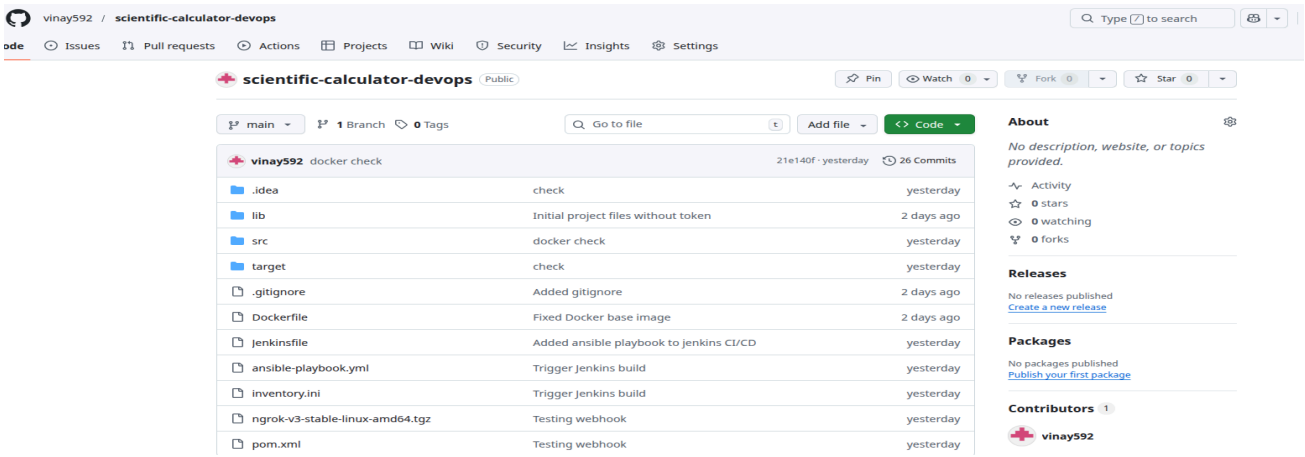
Notifications: Email (Jenkins Email Extension)

Source Code Management

Software Configuration Management (SCM) is a process used to control and monitor changes made to the source code during software development. It maintains a history of all updates and modifications made to the project files over time. SCM is especially useful when several developers are working on the same project, as it helps organize and combine their changes while avoiding conflicts. This process is commonly referred to as **Version Control**.

In a version control system, the project files are stored in a **repository**, which can be located either on a developer's local computer or on a remote server. The **local repository** is the working copy of the project on the developer's machine where code can be edited, tested, and updated before sharing it with others. To transfer the project from the local repository to the **remote repository**, a set of commands is executed to upload and synchronize the code.

- git init
- git add .
- git status
- git commit –m "commit message"
- git remote set-url origin git@github.com:vinay592/scientific-calculator-devops.git
- ssh -T git@github.com //Successfully Authenticated
- git push



Source Code Management

Jenkins:

Jenkins is an open-source automation server used to implement Continuous Integration and Continuous Deployment in software development. It helps automate tasks such as building, testing, and deploying applications. Jenkins can integrate with many development tools, which makes it useful for managing automated workflows in different environments.

In a CI/CD process, Jenkins monitors the GitHub repository for any new code changes. Whenever developers push a commit, Jenkins automatically triggers a pipeline that executes a sequence of predefined stages. This ensures that the application is built and prepared for deployment without manual intervention.

In this project, the Jenkins pipeline performs several automated steps. It pulls the latest code from GitHub, builds the project using Maven, creates a Docker image, pushes the image to Docker Hub, and finally runs an Ansible playbook to deploy the application.

Create CI Pipeline

A Jenkins pipeline is an automated workflow used to manage the process of building, testing, and deploying software directly from a version control system. It defines a sequence of stages that the code must pass through before it is released to the target environment. By automating these steps, the pipeline ensures that the development process is consistent, reliable, and less prone to human errors.

The pipeline is divided into several stages that perform different tasks in the workflow. These stages include checking out the latest source code from the repository, building the Docker image for the application, pushing the created image to Docker Hub, and executing an Ansible playbook to deploy the application. Each stage performs a specific function to move the project closer to deployment.

After all the stages are successfully completed, a post action is executed. This step sends an email notification to inform the user about the result of the pipeline, whether it has completed successfully or if any stage has failed. This notification helps developers quickly understand the status of the build and deployment process.

```
pipeline
{
    agent any
    options {
       disableConcurrentBuilds()
    }
    stages
    {

    stage('Checkout Code')
        {
            steps
            {
                checkout scm
            }
        }
        stage('Compile Program')
        {
            steps
            {
                sh 'javac src/program/Calculator.java'
            }
        }

        stage('Compile Test')
        {
            steps
            {
                sh 'javac -cp ".:src/program:lib/*" src/test/CalculatorTest.java'
            }
        }

        stage('Run Unit Tests')
        {
            steps
            {
                sh 'java -cp ".:src/program:src/test:lib/*" org.junit.runner.JUnitCore CalculatorTest'
            }
        }
```

```
        stage('Build Docker Image')
        {
            steps
            {
                sh 'docker build -t scientific-calculator .'
            }
        }
    stage('Push Docker Image')
    {
            steps
            {
                sh 'docker tag scientific-calculator vinayvb18/scientific-calculator'
                sh 'docker push vinayvb18/scientific-calculator'
            }
    }
    stage('Deploy using Ansible')
{
    steps
    {
        sh 'ansible-playbook -i inventory.ini ansible-playbook.yml'
    }
}
    }
    post
    {
       always
       {
        emailext(
            to: 'vinayvb592@gmail.com',
            subject: "Build Status: ${currentBuild.currentResult}",
            body: "Job: ${env.JOB_NAME}\nBuild Number: ${env.BUILD_NUMBER}\nStatus: ${currentBuild.currentResult}
        )
       }
    }

}
```

Stages of Pipeline

Configuration of Project in Jenkins

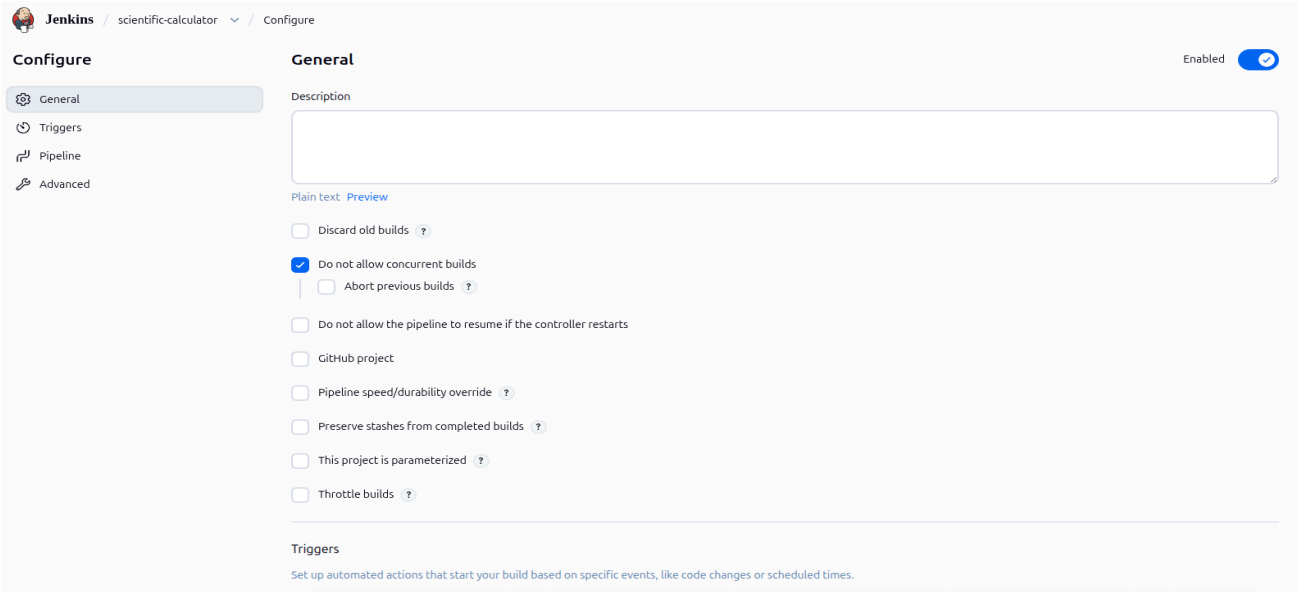
To run Jenkins in local machine there is a command to run it

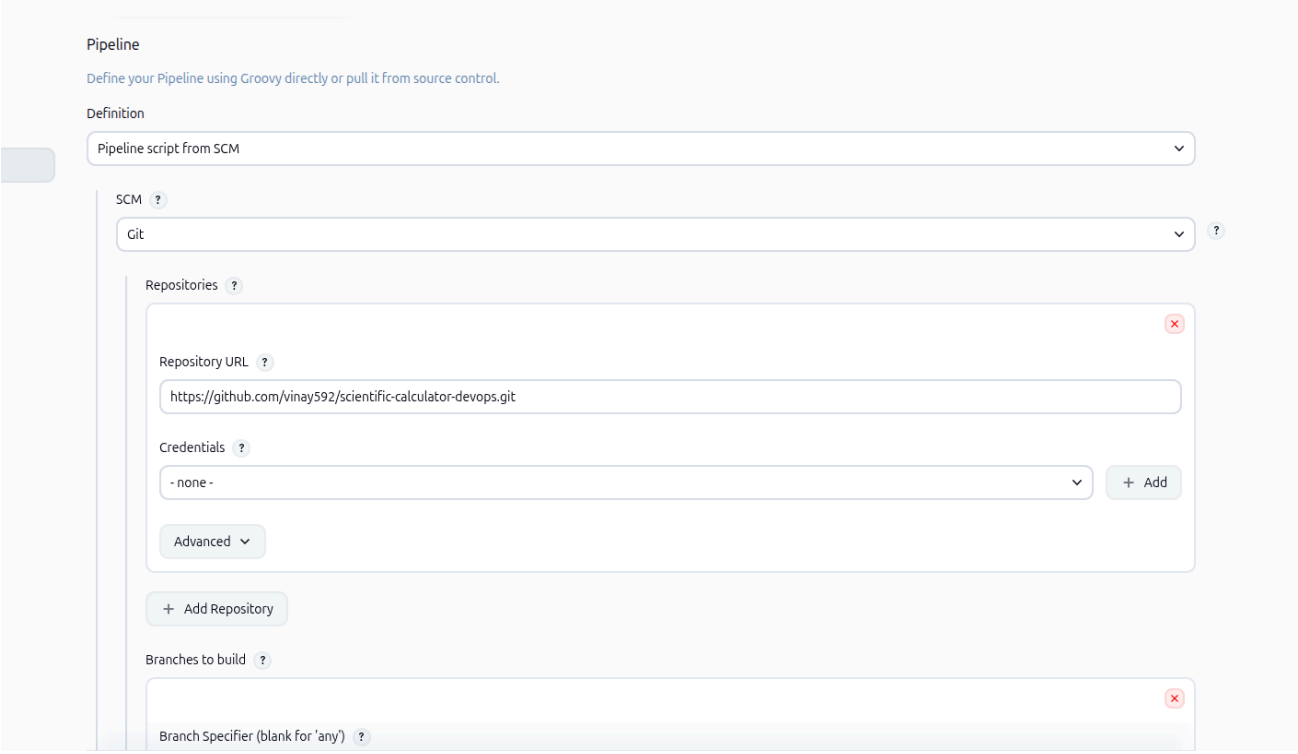
sudo systemctl start jenkins

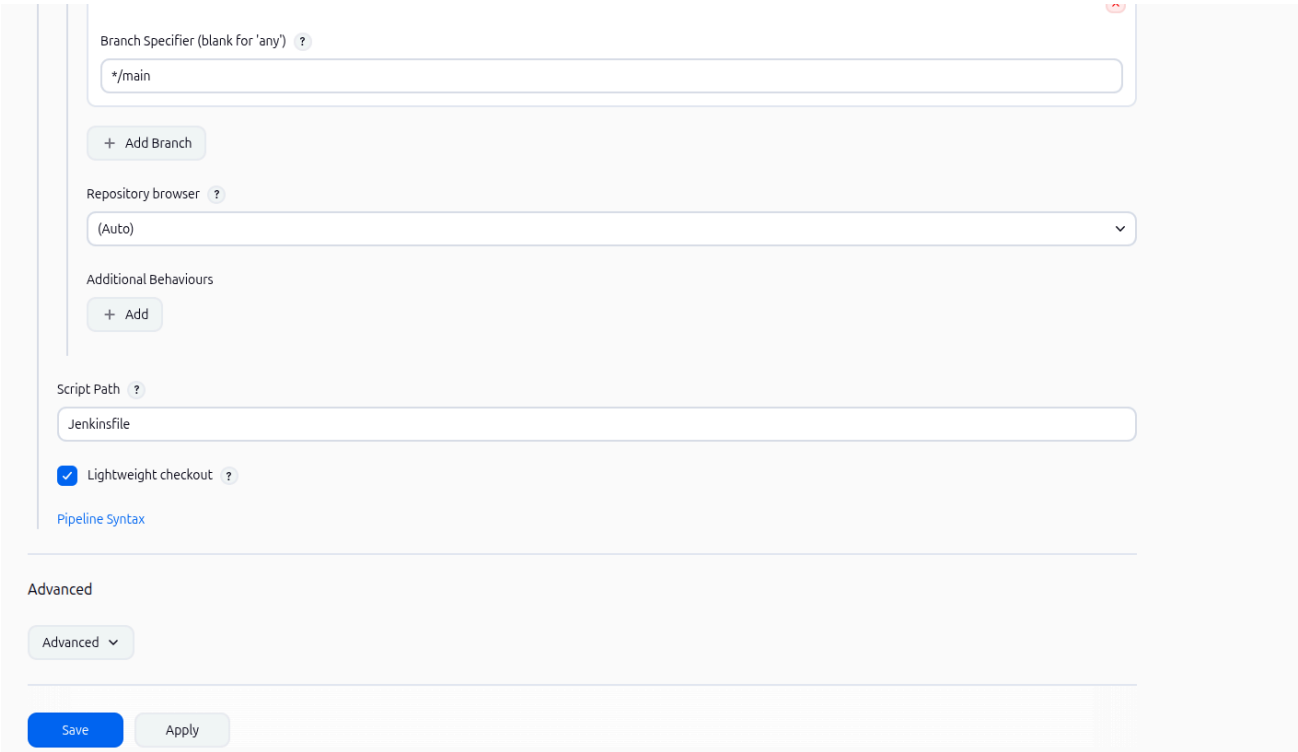
To check status of Jenkins:
sudo systemctl status jenkins

- Chrome =>localhost:8080

- Enter Username and Password
- New item
- Create new pipeline with some name







Jenkins Configuration

Here, we created a GitHub project in Jenkins and provided the URL of our GitHub repository. We also specified the path of the Jenkinsfile in the project configuration.

Next, we set system credentials in Jenkins for Docker Hub and the local machine used by the Ansible playbook. Since the project runs on the local machine and pulls the Docker image from Docker Hub, these credentials need to be configured in Jenkins.

Two credentials are added in Jenkins: one for the Docker Hub repository and another for the local machine. To set Docker Hub credentials, we provide the Docker Hub username from which the Docker image will be pulled.

- sudo su - jenkins
- Docker login // added credentials and password of dockerhub
- Store in /var/lib/jenkins/.docker/config.json

Docker:

Docker is a platform used to package applications along with their required dependencies into containers using operating system–level virtualization. In this project, a Docker image is created to run the **JAR file** that is generated after building the project with Maven. Once the image is created, it is uploaded to **Docker Hub**, which allows other systems to download the image and run the application inside containers.

The Docker image can be built using the following command:

```
docker build -t <USERNAME>/<IMAGE_NAME>:<TAG> .
```

After the image is created, it can be uploaded to Docker Hub using:

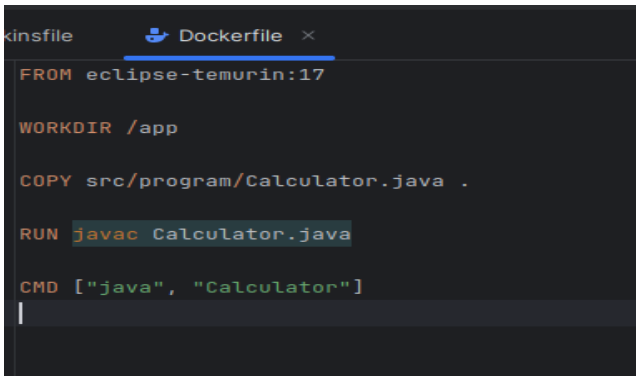
```
docker push <USERNAME>/<REPOSITORY_NAME>:<TAG>
```

In the CI/CD workflow, Jenkins pushes the Docker image to Docker Hub after the build process is completed. Later, the image is pulled on the target system through an Ansible job for deployment.

To run the container from the Docker image, the following command can be used:

```
docker run -it <image_name>
```

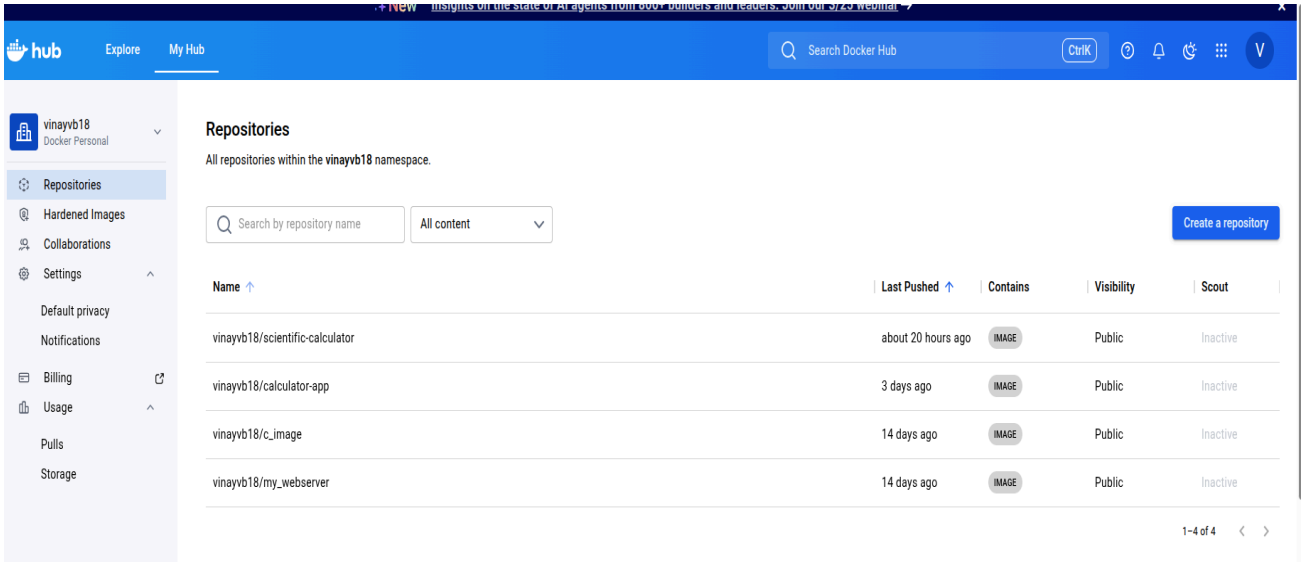
After pushing the image successfully, it can be viewed and managed from the Docker Hub profile



DockerFile

To build the Docker image and push it to the Docker Hub remote repository, the required steps are defined in the Jenkins pipeline script.

The Docker image is created and uploaded to the user's Docker Hub repository. After the image is successfully pushed, the local copy of the image on the machine is deleted.



Docker Hub Repository

Ansible:

Ansible is an automation tool used for deploying and managing applications across multiple systems. In its architecture, the machine that runs Ansible is called the **control node**, and the systems where tasks are executed are known as **managed nodes**.

Only the control node requires Ansible to be installed. It connects to the managed nodes using **SSH** to perform operations remotely.

The tasks that need to be executed are written in a **YAML (.yml) file** called a **playbook**. Information about the managed nodes is stored in an **inventory file**.

The playbook can be executed using the following command:

ansible-playbook <playbook> -i <inventory>

```
[app]
localhost ansible_connection=local
```

Inventory file

The **Ansible inventory file** lists the hosts and host groups on which the playbook tasks will run. It provides details about the managed nodes that are controlled by Ansible. The inventory file can be written in different formats depending on the configuration of the Ansible environment.

Ansible playbooks define the automation tasks used for configuration and deployment. They contain a set of instructions that Ansible executes on the target systems.

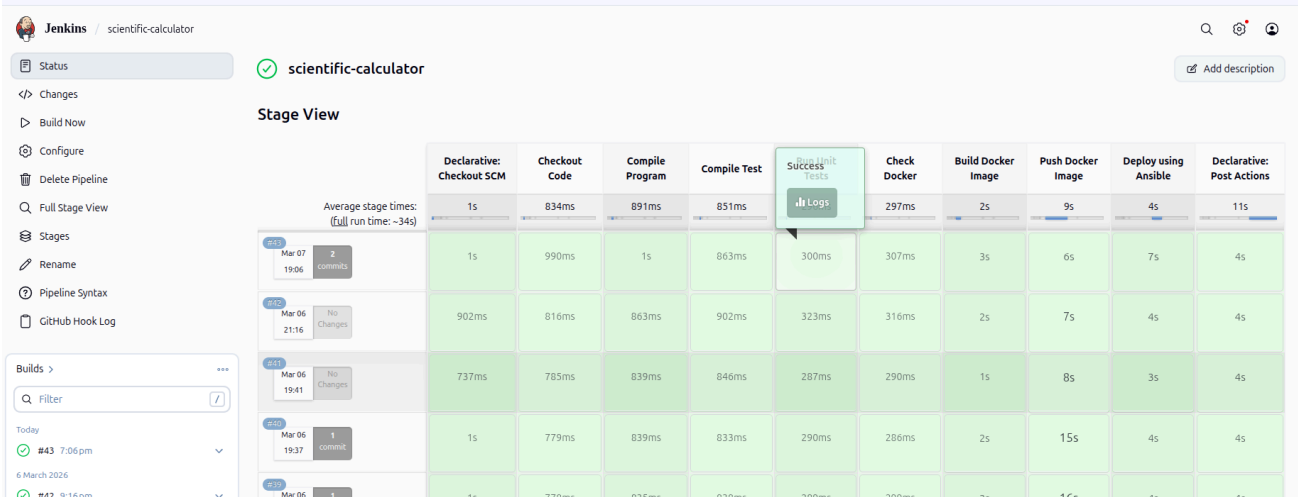
To run the application's Docker image on the host system, required tools such as **Docker and Python pip** must be installed. The playbook automates this setup, pulls the Docker image from Docker Hub, and launches the container on the specified hosts.

```yaml
---
- name: Deploy Scientific Calculator Container
  hosts: local
  become: true

  tasks:

    - name: Run Calculator Container
      docker_container:
        name: scientific-calculator
        image: vinayvb18/scientific-calculator
        state: started
        recreate: yes
        tty: true
        pull: yes
        interactive: true
```

Playbook file

Run Project as Pipeline in Jenkins

To run the project as pipeline in jenkins we have to click build now to see each stages of pipeline and show the output of each stages.



Source Code of Scientific-Calculator

Below is the source code for scientific-calculator

```java
import java.util.Scanner;

public class Calculator
{
    public static double squareRoot(double n) { return Math.sqrt(n); }

    public static int factorial(int n)
    {
        int fact = 1;

        for(int i=1;i<=n;i++)
            fact = fact * i;

        return fact;
    }

    public static double naturalLog(double n) { return Math.log(n); }

    public static double power(double a,double b) { return Math.pow(a,b); }

    public static double add(double a,double b) { return a+b; }

    public static double subtract(double a,double b) { return a-b; }

    public static double multiply(double a,double b) { return a*b; }

    public static double divide(double a,double b) { return a/b; }

    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        while(true)
        {
            System.out.println();
            System.out.println("1 Square Root");
            System.out.println("2 Factorial");
            System.out.println("3 Natural Log");
            System.out.println("4 Power");
            System.out.println("5 Addition");
```

```java
            System.out.println("5 Addition");
            System.out.println("6 Subtraction");
            System.out.println("7 Multiplication");
            System.out.println("8 Division");
            System.out.println("9 Exit");

            System.out.print("Enter your choice : ");

            int choice = sc.nextInt();

            if(choice==1)
            {
                System.out.print("Enter number : ");
                double n = sc.nextDouble();

                System.out.println("Result : "+squareRoot(n));
            }

            else if(choice==2)
            {
                System.out.print("Enter number : ");
                int n = sc.nextInt();

                System.out.println("Result : "+factorial(n));
            }

            else if(choice==3)
            {
                System.out.print("Enter number : ");
                double n = sc.nextDouble();

                System.out.println("Result : "+naturalLog(n));
            }

            else if(choice==4)
            {
                System.out.print("Enter first number : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
```

```java
                System.out.print("Enter first number : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
                double b = sc.nextDouble();

                System.out.println("Result : "+power(a,b));
            }

            else if(choice==5)
            {
                System.out.print("Enter first number : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
                double b = sc.nextDouble();

                System.out.println("Result : "+add(a,b));
            }

            else if(choice==6)
            {
                System.out.print("Enter first number : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
                double b = sc.nextDouble();

                System.out.println("Result : "+subtract(a,b));
            }

            else if(choice==7)
            {
                System.out.print("Enter first number : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
                double b = sc.nextDouble();

                System.out.println("Result : "+multiply(a,b));
```


```java
                System.out.print("Enter first number : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
                double b = sc.nextDouble();

                System.out.println("Result : "+multiply(a,b));
```


```java
            }

            else if(choice==8)
            {
                System.out.print("Enter first number : ");
                double a = sc.nextDouble();

                System.out.print("Enter second number : ");
                double b = sc.nextDouble();

                System.out.println("Result : "+divide(a,b));
            }

            else
                break;
        }
    }
}
// test webhook
```

Adding Test Cases:

What is Unit Testing?

- Unit Testing is the process of testing individual components of code in isolation.
- A "unit" refers to the smallest functional part of a system, which could be a single line of code, a method, or an entire class.
- The smaller the unit being tested, the faster and more efficient the process becomes.
- This helps in gaining deeper insights into the code's functionality and overall performance.

JUnit: A Unit Testing Framework for Java.

- JUnit is an open-source framework designed for unit testing in Java.
- It allows developers to write and execute automated test cases efficiently.
- JUnit also provides visual feedback on test execution through graphical indicators. By leveraging JUnit, developers can create robust, error-free code with greater reliability.

```java
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest
{
    @Test
    public void testSquareRoot() { assertEquals(4.0, Calculator.squareRoot(16), 0.001); }

    @Test
    public void testFactorial() { assertEquals(120, Calculator.factorial(5)); }

    @Test
    public void testNaturalLog() { assertEquals(1.0, Calculator.naturalLog(Math.E), 0.001); }

    @Test
    public void testPower() { assertEquals(8.0, Calculator.power(2,3), 0.001); }
}
```

Test Cases

Webhooks:

- **Webhooks** are automated HTTP callbacks that trigger an action when a specific event occurs, such as pushing code to a GitHub repository.
- They help connect different services so that tasks like builds and deployments start automatically without manual execution.

Using Webhooks with Jenkins and Ngrok

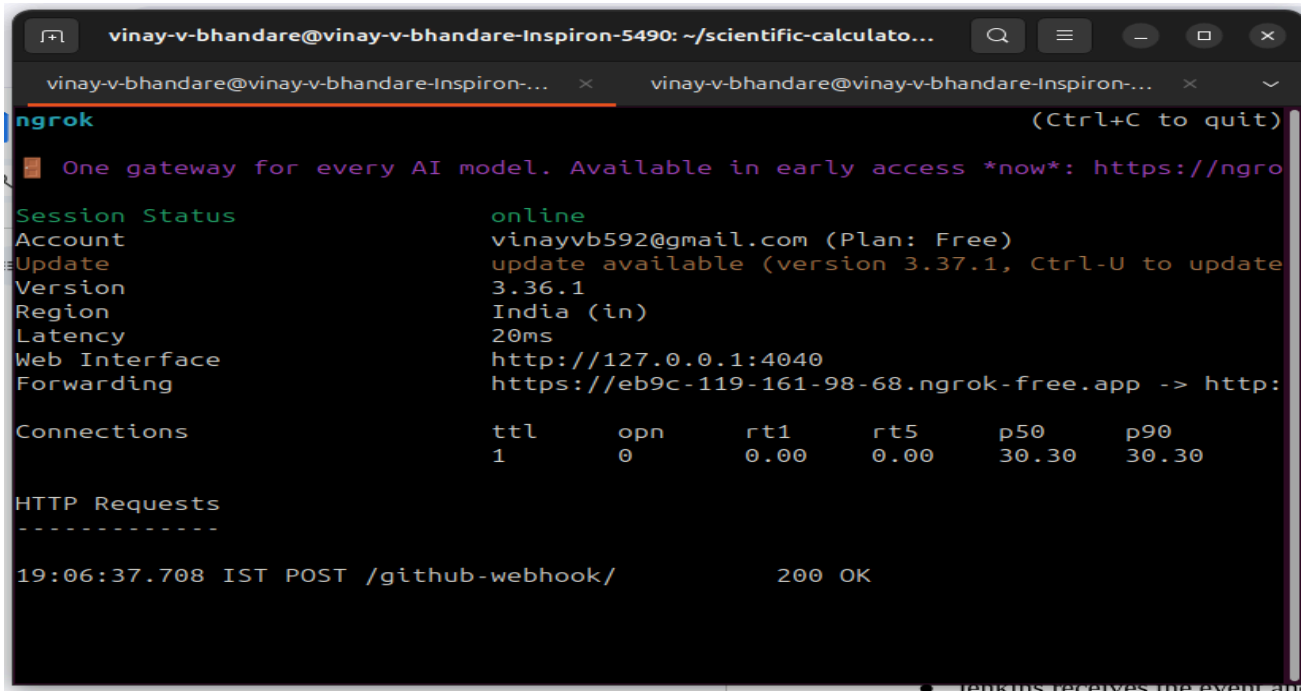
- When Jenkins is running on a **local machine**, it cannot normally receive webhook requests from GitHub because it is not accessible from the internet.
- To solve this problem, **Ngrok** is used. Ngrok creates a **secure public URL** that tunnels traffic from the internet to the local Jenkins server.

Ngrok Installation and Setup

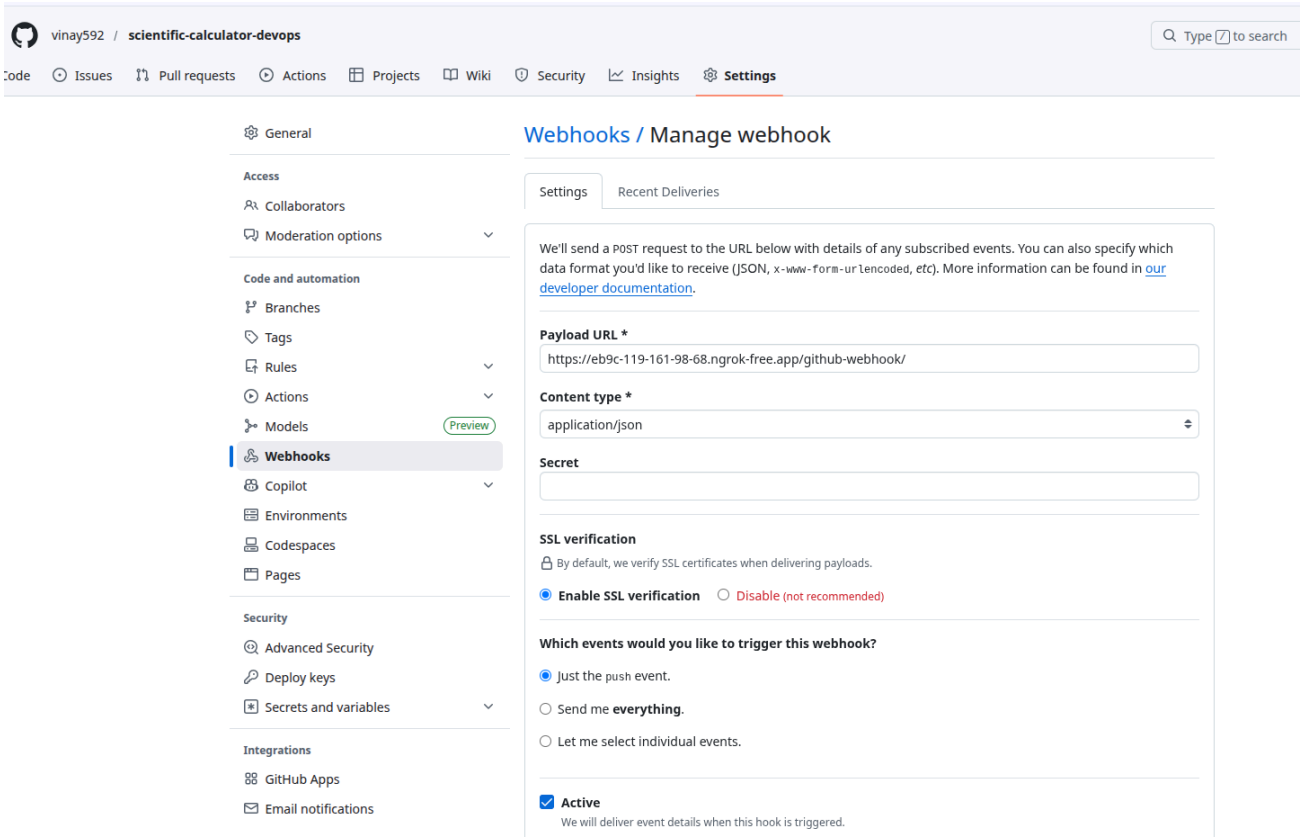
- First, download **Ngrok** from the official website and extract it on the system.
- After installation, run the command ngrok http 8080 (assuming Jenkins runs on port 8080).
- Ngrok generates a **public URL** that forwards requests to the local Jenkins server.
- This public URL is then added as a **webhook URL in the GitHub repository settings**.

Working Process

- Whenever code is pushed to the GitHub repository, the webhook sends a request to the Ngrok public URL.
- Ngrok forwards this request to the local Jenkins server.
- Jenkins receives the event and **automatically triggers the build pipeline**.



Ngrok http 8080



Webhook Configuration

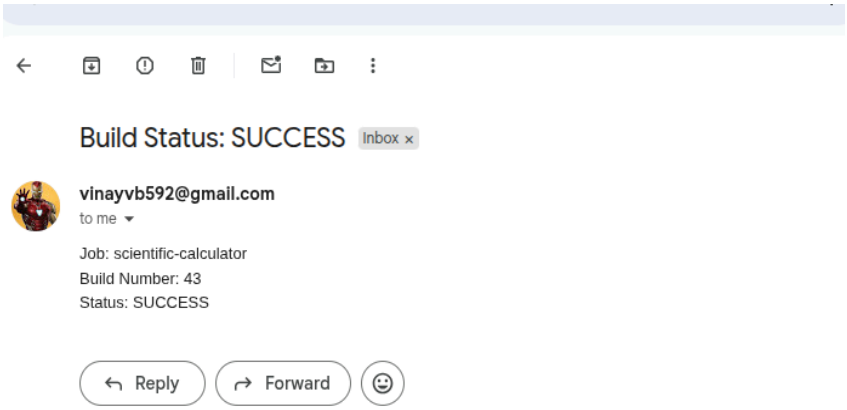
Email Notification in Jenkins Pipeline Project and docker run

What is SMTP in Jenkins?

- **SMTP (Simple Mail Transfer Protocol)** is used in Jenkins to send email notifications regarding the status of builds and pipelines.
- It allows Jenkins to inform developers or team members whether a build has succeeded or failed.
- This helps keep everyone updated about the progress of the project.

Use of SMTP in Jenkins

- **Automated email alerts:** Jenkins automatically sends emails after each pipeline execution.
- **Failure notifications:** Developers are notified immediately if a build fails so that issues can be fixed quickly.
- **Custom email messages:** Emails can include build logs, reports, and other important details related to the pipeline execution.



Email Sent By Jenkins after successful build

Project Runs successfully and now our docker image is also pulled from docker hub. We can run in local machine to check whether it runs or not by using below command:

docker exec -it scientific-calculator java Calculator

```
Enter your choice : 9
vinay-v-bhandare@vinay-v-bhandare-Inspiron-5490:~/scientific-calculator-devops$
docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATE
D          STATUS          PORTS     NAMES
8d2561d9be63   vinayvb18/scientific-calculator   "/__cacert_entrypoin…"   39 min
utes ago   Up 39 minutes             scientific-calculator
vinay-v-bhandare@vinay-v-bhandare-Inspiron-5490:~/scientific-calculator-devops$
docker exec -it scientific-calculator java Calculator

1 Square Root
2 Factorial
3 Natural Log
4 Power
5 Addition
6 Subtraction
7 Multiplication
8 Division
9 Exit
Enter your choice : 2
Enter number : 5
Result : 120
```

Project running successfully on local machine

Github Repo:
https://github.com/vinay592/scientific-calculator-devops

Docker Hub Repo:
https://hub.docker.com/repository/docker/vinayvb18/scientific-calculator/general